Ex. no: 06 Date: 8-8-2025

Error detection and correction using hamming code.

Aim:

To write a program to implement error detection and correction using the hamming code.

Algorithm:

1) start the program.

*) Read the number of 4 bit data blocks (n) forming the input data stream

* for each 4 bit data block D = d_1 d_2 d_3 d_4, perform hamming encoding:

a) Arrange 7 bits with a parity bits at position 1, 2, and 4 and data bits at positions 3, 5, 6 and 7

b) compute the parity bits using even parity:

$P_1 = d_1$ XOR d_2 XOR d_4 (covers bit pos 1, 3, 5, 7)

$P_2 = d_1$ XOR d_3 XOR d_4 (covers bit pos 2, 3, 6, 7)

$P_4 = d_2$ XOR d_3 XOR d_4 (covers bit pos 4, 5, 6, 7)

c) From the 7-bit hamming code word: $P_1 P_2 d_1$ $P_4 d_2 d_3 d_4$

* simulate the transmission by introducing a single bit error at a choosen bit position in the code word

```
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]

    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]

    error_pos = c1*1 + c2*2 + c4*4

    if error_pos != 0:
        print("Error detected at the position:", error_po
        print("Correcting....")
        h[error_pos-1] ^= 1

    else:
        print("No error detected")

    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)
```

```
def main():
    print("-- HAMMING CODE - ERROR REDUCTION AND CORRECT
    n = int(input("Enter no of 4-bit data block in the
    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
        d = list(map(int, input("Enter 4 data bits {
            (space seperated, e.g 1 0 1 1): ").split()))

        h = encode(d)
        print(f"Original data bits : {to-str(d)}")
        print(f"Encoded Hamming Code (7 bits): {to-str(
        pos = int(input("Enter bit position to intro
error (1-7), 0 = no error): "))
        if (1 <= pos <= 7):
            h[pos-1] ^= 1
        print(f"Transmitted code (with error
included at pos {pos}: {to-str(h)}")
        else:
            print(f"Transmitted code (no error
                induced): {to-str(h)}")
        print(f"Recived code : {to-str(h)}")
        print("checking for error")
        detect_and_correct(h)
```

```
    print(f"Recovered data bits : {to_str(rd)}")

..\n")

))

if __name__ == "__main__":
    main()
```

Output

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION===

Enter number of 4 bit data block in the stream : 2

--- Block 1 ----

Enter 4 data bits (space separated, eg. 1011): 1011

Original data bits : 1011

Encoded Hamming code (7 bits) : 0110011

Enter bit position to introduce error (1-7) 0 =(no error): 3

Transmitted code( with error induced at pos 3): 0100011

Recieved code : 0100011

checking for errors.....

    Error detected at bit position : 3 → correcting..

corrected code : 0110011

Recovered data bits : 1011

Block 2 ----

Enter 4 data bits (space seperated, eg :1011) : 0110

Original data bits : 0110

Encoded Hamming Code (7 bits) : 1100110
```

Enter bit position to introduce error (1-7, 0 = no error): 6

Transmitted code (with error induced at pos 6): 1100100

Received code : 1100100

checking for errors : ...

Error detected at bit position : 6 → correcting

corrected code : 1100110

recovered data bits : 1110

Result :

The program to implement error detection and correction using the hamming code concept was successfully written in python, executed and verified using a sample input data stream.